

Лабораторная работа 1.

Основы PyTorch и обучение полносвязной нейронной сети

В этой лабораторной работе вы пройдёте все этапы создания и обучения нейронной сети на Python с использованием библиотеки PyTorch. Ваша цель — научиться работать с тензорами, понять принцип автоматического вычисления градиентов и создать простую полносвязную нейронную сеть для классификации табличных данных.

1. Подготовка данных

Загрузка датасета

Для выполнения лабораторной работы необходимо использовать **табличный датасет задачи классификации**. Рекомендуется использовать датасет из Лабораторной работы №3 по СтатОИВ, так как это позволит выполнить корректное сравнение различных подходов на одних и тех же данных. Допускается использование другого датасета из библиотеки `sklearn.datasets` или сайта `kaggle` для задачи классификации объёмом не более 50 000 объектов и не более 100 признаков, не требующего сложной предобработки данных. Использование изображений, текстов и других неструктурированных данных в данной лабораторной работе не допускается.

Разделение данных

Разделите датасет на обучающую и тестовую выборки. Используйте функцию `train_test_split` из библиотеки `sklearn`. Рекомендуемое соотношение: 80% на обучение, 20% на тестирование.

Нормализация признаков

Выполните стандартизацию признаков с помощью `StandardScaler` из `sklearn`. Это необходимо для того, чтобы все признаки имели сопоставимые

масштабы, что улучшит качество обучения.

Преобразование в тензоры

Преобразуйте подготовленные данные (признаки и метки классов) в тензоры PyTorch.

2. Создание нейронной сети

Архитектура сети

Создайте класс нейронной сети, наследуясь от `nn.Module`. Ваша сеть должна включать как минимум:

- **Входной слой** — размерность соответствует числу признаков в датасете
- **Скрытый слой** — выберите количество нейронов. Рекомендуется начать с 32 или 64 нейронов, после чего при желании проверить несколько альтернативных значений и сравнить качество модели на тестовой выборке.
- **Выходной слой** — количество нейронов равно числу классов в задаче

Функция активации

После скрытого слоя нейронной сети необходимо использовать **нелинейную функцию активации**. Без нелинейности даже многослойная сеть сводится к одному линейному преобразованию и не способна моделировать сложные зависимости в данных.

В данной лабораторной работе в качестве базового варианта используется функция **ReLU (Rectified Linear Unit)** вычисляется как `max(0, x)`, проста в вычислении и ускоряет обучение, снижает проблему затухающих градиентов.

При этом важно понимать, что ReLU — не единственная возможная функция активации. В нейронных сетях также применяются:

- **Sigmoid** — отображает значения в диапазон от 0 до 1, удобна для вероятностей, но вызывает затухание градиентов в глубоких сетях
- **Tanh** — имеет выходы в диапазоне от -1 до 1 и нулевое среднее, обучается стабильнее сигмоиды, но также подвержена затухающим градиентам

- **Leaky ReLU, ELU и другие модификации ReLU** — сохраняют небольшой градиент при отрицательных входах и уменьшают риск «замирания» нейронов

В рамках лабораторной работы достаточно использовать **ReLU**, однако при желании можно провести небольшой эксперимент и сравнить влияние разных функций активации на скорость обучения и качество классификации. Это позволит увидеть, что выбор функции активации является важным архитектурным решением нейронной сети.

Функция потерь и оптимизатор

Для обучения нейронной сети необходимо определить **функцию потерь** и **алгоритм оптимизации параметров модели**. Функция потерь показывает, насколько предсказания сети отличаются от истинных меток классов, а оптимизатор использует эту информацию для обновления весов сети в процессе обратного распространения ошибки.

В задаче многоклассовой классификации используется функция потерь

```
nn.CrossEntropyLoss()
```

В качестве алгоритма оптимизации применяется `torch.optim.Adam()` — один из наиболее распространённых методов градиентного спуска в глубоком обучении.

Важно понимать, что существуют и другие варианты:

- **SGD (стохастический градиентный спуск)** — более простой и интерпретируемый метод, часто используется в научных работах и при обучении крупных моделей
- **RMSprop, AdamW и другие модификации** — применяются в более сложных архитектурах и позволяют лучше контролировать процесс обучения

В рамках данной лабораторной работы достаточно использовать связку `CrossEntropyLoss + Adam`, однако в дальнейшем курсе вы познакомитесь с различными стратегиями оптимизации и их влиянием на качество и скорость обучения нейронных сетей.

3. Обучение модели

Цикл обучения

Реализуйте процесс обучения нейронной сети на протяжении **не менее 50 эпох**.

В каждой эпохе необходимо последовательно выполнить следующие действия на обучающей выборке:

1. Обнулить накопленные градиенты параметров модели с помощью `optimizer.zero_grad()`, так как в PyTorch градиенты суммируются между итерациями.
2. Выполнить **прямой проход (forward pass)**: передать входные признаки в модель и получить выход сети.
3. Вычислить значение **функции потерь**, сравнив предсказания модели с истинными метками классов.
4. Выполнить **обратное распространение ошибки** вызовом `loss.backward()`, в результате чего будут вычислены градиенты по всем обучаемым параметрам сети.
5. Обновить веса модели с помощью шага оптимизации `optimizer.step()`, используя ранее вычисленные градиенты.

Отслеживание метрик

Во время обучения необходимо фиксировать метрики на каждой эпохе, чтобы отслеживать процесс сходимости и выявлять возможное переобучение. Следует сохранять:

- **Функцию потерь на обучающей выборке** — вычисляется после обновления параметров сети.
- **Точность (ассигасу) на обучающей выборке** — доля правильных предсказаний; для многоклассовой задачи используйте `torch.argmax(output, dim=1)` для получения предсказанного класса.
- **Функцию потерь на тестовой выборке** — рассчитывается в режиме `torch.no_grad()`, чтобы не хранить вычислительный граф.
- **Точность (ассигасу) на тестовой выборке** — также вычисляется без градиентов.

Сохранённые значения метрик можно использовать для построения графиков изменения функции потерь и точности по эпохам, что позволит

визуально проанализировать качество обучения модели и признаки переобучения.

Визуализация результатов

Постройте графики с помощью библиотеки `matplotlib/seaborn` :

- График изменения функции потерь (train и test на одном графике)
- График изменения точности (train и test на одном графике)

4. Сравнение с результатами прошлого семестра

В этой части лабораторной работы вы сравниваете эффективность нейронной сети с результатами, которые были получены на том же датасете в прошлом семестре с использованием классических методов машинного обучения (логистическая регрессия, деревья решений, k-ближайших соседей, градиентный бустинг и т.д.).

Возьмите лучший результат из прошлого семестра

Определите, какой классический алгоритм показал наибольшую `accuracy` , `F1-score` на тестовой выборке. Запишите значения метрик.

Посчитайте метрики полученной нейронной сети

Сохраняйте метрики на обучающей и тестовой выборке, включая `accuracy` и `F1-score` .

Сравните результаты по метрикам

Для честного сравнения используйте одинаковое разбиение на обучающую и тестовую выборки. Составьте таблицу или график, где указаны значения accuracy и F1 для классического метода и для нейронной сети.

Объясните **почему** результаты различаются. Подумайте о сложности модели, количестве данных, нелинейности признаков и возможном переобучении.